

DICOM Overlay & Masking Guide

Theory, Attributes, Legacy vs. Modern Implementation, and Commercial Viewer Practice

DICOM / PACS Training Series

September 2026

1. Why Do We Mask / Overlay at All?
2. Types of Overlays — The Taxonomy
 - 2.1 Graphic Overlay (ROI / “G” type Overlay Plane)
 - 2.2 ROI Overlay (“R” type Overlay Plane)
 - 2.3 Shutter Overlay (Bitmap Shutter)
 - 2.4 Mask (Subtraction) Overlay
 - 2.5 Burned-In Overlay / Burned-In Annotation
 - 2.6 Anonymization / De-identification Masking
3. Legacy vs. Modern: The Big Picture
4. The Overlay Plane Module (Group 60xx) — Deep Dive
 - 4.1 Encoding Mechanics
 - 4.2 Overlay Plane Module — Attribute Table (PS3.3 §C.9.2, C.9.3)
 - 4.3 Related Attribute — Bitmap Shutter Linkage
5. The Mask Module (Subtraction) — Deep Dive
 - 5.1 The Clinical Problem It Solves
 - 5.2 Where the Module Lives
 - 5.3 Mask Module — Attribute Table (PS3.3 §C.7.6.10, Table C.7-16)
 - 5.4 Mask Operation — Defined Terms Explained Simply
 - 5.5 Practical Walkthrough (AVG_SUB Example)
6. Where Overlay/Mask Sit in the Rendering Pipeline
7. Burned-In Annotation vs. Stored (Removable) Overlay
 - 7.1 Key Attribute for This Distinction
 - 7.2 Anonymization / De-identification Overlay Masking in Practice
8. How Commercial PACS/Diagnostic Viewers Actually Handle This
9. Use Cases Summary
10. Code Examples
 - 10.1 Reading a Classic Overlay Plane with `pydicom`
 - 10.2 Interpreting the Mask Module with `pydicom`
 - 10.3 Minimal Manual Subtraction (AVG_SUB) in NumPy
 - 10.4 Reading an Overlay with `fo-dicom` (C#/.NET)
 - 10.5 Checking Burned-In Annotation Before De-identification (`pydicom`)
11. Quick-Reference Comparison: Legacy vs. Modern
12. References

1. Why Do We Mask / Overlay at All?

A DICOM image is, at its core, a matrix of pixel intensities. Clinicians rarely want *just* that matrix — they want it annotated, highlighted, cropped to a usable region, stripped of distracting background anatomy, or stripped of identifying text. “Overlay” and “mask” are the two DICOM mechanisms that let you add or subtract information **without touching the original pixel data**.

Need	Mechanism used
Draw a cursor, graticule, biopsy marker, or free-text label on top of an image	Graphic Overlay
Highlight a Region of Interest (ROI) — e.g., a lesion boundary	ROI Overlay
Show only the true image area on an ultrasound cone/sector display, hiding scan-converter padding	Bitmap Shutter (Overlay-based)
Remove stationary anatomy/bone from a vascular run so only contrast-filled vessels remain	Mask Subtraction (DSA)
Hide/replace patient-identifying pixels that were burned into the image	Anonymization / De-identification overlay (masking)
Permanently stamp annotations into the pixel data for legal/archival reasons	Burned-in annotation

The core principle: DICOM was designed so that diagnostic pixel data and *presentation* information (what to draw on top, what to subtract, what to hide) could travel either **together** (legacy, embedded) or **separately** (modern, non-destructive). Understanding *which* era of the standard you are looking at explains almost every quirk you will hit in a PACS/VNA or viewer implementation.

2. Types of Overlays — The Taxonomy

DICOM overlays are not one thing; the word is used loosely in clinical/vendor conversation to mean several distinct standard mechanisms. Keep these separate:

2.1 Graphic Overlay (ROI / “G” type Overlay Plane)

A 1-bit-deep bitmap, same pixel grid as the image (or a sub-region of it), drawn as graphics — arrows, calipers, free text rendered as pixels, biopsy needle paths. Defined by the **Overlay Plane Module** (Section 4).

2.2 ROI Overlay (“R” type Overlay Plane)

Same module, but `Overlay Type = R`. Used to encode a Region-of-Interest mask (e.g., a segmented tumor boundary) as a binary bitmap layered on the image, historically used before Segmentation IOD existed.

2.3 Shutter Overlay (Bitmap Shutter)

A special use of a group-60xx overlay whose *purpose* is to define the collimation/active image area (e.g., the round or sector-shaped active area of an ultrasound or angio image), referenced via **Shutter Overlay Group (0018,1623)**. Distinguishes real acquired pixels from black padding around them.

2.4 Mask (Subtraction) Overlay

Not a bitmap overlay at all — a **computational instruction set** (Mask Module) that tells the viewer *how to subtract one frame from another* to suppress static background anatomy. The centerpiece of Digital Subtraction Angiography (DSA). Covered in depth in Section 5.

2.5 Burned-In Overlay / Burned-In Annotation

Overlay or annotation information that has been **permanently composited into the Pixel Data** itself (not stored separately and removable). Signalled by attribute **Burned In Annotation (0028,0301) = YES**. Common on ultrasound, ECG traces, secondary captures, and some modality consoles that print patient name/ID/timestamp directly onto the image.

2.6 Anonymization / De-identification Masking

Not a DICOM-defined object type but a **workflow use of the above**: either (a) detecting and blacking-out/blurring burned-in PHI pixels (image-processing based redaction), or (b) removing/zeroing Overlay Data elements and associated PHI-bearing overlay text during de-identification per **PS3.15** (De-identification profiles). This is the single biggest practical headache in enterprise PACS/VNA migrations.

Overlay Type	Standard Home	Stored As	Removable Non-Destructively?
Graphic (G)	Overlay Plane Module (60xx)	Separate bitmap element	Yes
ROI (R)	Overlay Plane Module (60xx)	Separate bitmap element	Yes
Shutter	Overlay Plane Module (60xx) + (0018,1623)	Separate bitmap element	Yes
Mask/Subtraction	Mask Module / XA-XRF Mask Module	Instructions only, no bitmap	Yes (it’s metadata)
Burned-in annotation	N/A (pixel-level)	Inside Pixel Data (7FE0,0010)	No — requires pixel editing/redaction
Legacy embedded overlay	Retired usage of Pixel Data high bits	Inside Pixel Data unused bit-planes	No — requires bit-plane stripping

3. Legacy vs. Modern: The Big Picture

DICOM’s overlay story has three distinct generations. This is the single most important mental model for an architect working across old PACS and new AI-DLC pipelines.

Generation	Mechanism	Status	Typical Source	Risk/Limitation
Gen 1 — Embedded bit-plane overlay	Overlay stored in unused high bits of Pixel Data (7FE0,0010) when Bits Allocated=16 but Bits Stored=8/12	Retired (PS3.3/PS3.5 pre-2004)	Old CR/US/Angio consoles (1990s)	Invisible to modern parsers; silently lost on transcoding/compression; a major legacy-data land-mine
Gen 2 — Standalone Overlay Plane Module	Separate repeating group 60xx,eeee , Overlay Data (60xx,3000)	Current but discouraged for new SOP Classes; still legal, still seen constantly in the field (esp. US, XA, secondary capture)	1996-era standard through today; still emitted by many modality consoles	Only 16 overlay groups per instance (60xx where xx = 00,02,...,1E); undefined color/intensity rendering outside a Presentation State; not searchable/queryable; many modern viewers ignore or half-support it
Gen 3 — Structured, separated Information Objects	Grayscale/Color Softcopy Presentation State (GSPS/CSPS), Segmentation IOD, Parametric Map IOD, Structured Reports (SR) for annotations, Mask Module for DSA	Current, actively developed	PACS/AI post-processing, modern XA systems, AI segmentation output	Requires viewer support for the specific IOD; more complex object model; but non-destructive, queryable, and machine-parseable

Rule of thumb used in practice: if a graphic must be simple, per-instance, and rendered exactly as acquired (e.g., an US sector shutter or simple ROI), Gen-2 Overlay Plane Module is still perfectly valid and common. If a graphic needs clinical semantics, measurements, multi-viewer interoperability, or machine-readability (AI-generated contours, structured measurements), Gen-3 objects (GSPS, SEG, SR) are the modern, standards-preferred path.

4. The Overlay Plane Module (Group 60xx) — Deep Dive

4.1 Encoding Mechanics

- Up to **16 overlay planes** may accompany one image, addressed by repeating group numbers 6000, 6002, 6004, ... 601E (i.e., 60xx where xx is an even value 00–1E).
- Each overlay plane is **exactly 1 bit deep** — a pure binary mask (pixel is “on” or “off”).
- Overlay pixels are packed and stored in **Overlay Data (60xx,3000)**, historically with VR OW (Implicit VR Little Endian *requires* OW; explicit-VR transfer syntaxes may use OB).
- Overlay Rows / Overlay Columns may match the image matrix or be smaller (e.g., a single-frame overlay placed at an offset via Overlay Origin).
- Overlay Bits Allocated (60xx,0100)** is always 1, and **Overlay Bit Position (60xx,0102)** is always 0 for properly encoded (Gen-2) overlays — these attributes are vestiges of the retired Gen-1 embedded-bit-plane scheme, kept for backward parsing only.
- Rendering color/intensity of a plain Overlay Plane is **explicitly undefined** by the standard unless a Presentation State SOP Instance is controlling display — in practice, viewers hard-code overlays to a color (commonly green or the modality’s default) when no Presentation State is present.

4.2 Overlay Plane Module — Attribute Table (PS3.3 §C.9.2, C.9.3)

Attribute Name	Tag	Type	Description
Overlay Rows	(60xx,0010)	1	Number of rows in the overlay bitmap
Overlay Columns	(60xx,0011)	1	Number of columns in the overlay bitmap
Overlay Type	(60xx,0040)	1	G = Graphics, R = ROI
Overlay Origin	(60xx,0050)	1	Row\Column position of overlay’s first pixel relative to image (1\1 = image’s top-left pixel)
Overlay Bits Allocated	(60xx,0100)	1	Always 1 for current-usage overlays
Overlay Bit Position	(60xx,0102)	1	Always 0 for current-usage overlays
Overlay Data	(60xx,3000)	1	The packed bitmap itself (VR OW/OB)
Overlay Description	(60xx,0022)	3	Free-text description of the overlay’s purpose
Overlay Subtype	(60xx,0045)	3	Further categorizes an ROI-type overlay, e.g. USER, AUTOMATED
Overlay Label	(60xx,1500)	3	Short label for the overlay (e.g., “Biopsy Path”)
ROI Area	(60xx,1301)	3	Area (in pixels) of an ROI overlay
ROI Mean	(60xx,1302)	3	Mean pixel value within the ROI
ROI Standard Deviation	(60xx,1303)	3	Std. dev. of pixel values within the ROI
Number of Frames in Overlay	(60xx,0015)	1C	For multi-frame images, how many frames the overlay applies to
Image Frame Origin	(60xx,0051)	1C	Frame number the overlay starts at in a multi-frame image

(Note: xx ranges over 00,02,04,...,1E — sixteen possible repeating groups, one overlay per group per image.)

4.3 Related Attribute — Bitmap Shutter Linkage

Attribute Name	Tag	Purpose
Shutter Overlay Group	(0018,1623)	Points a Display Shutter Module to a specific 60xx group used purely to mask/shutter the active image area
Active Image Area Overlay Group	(0018,6070)	Similar linkage used in Ultrasound Region Calibration context

5. The Mask Module (Subtraction) — Deep Dive

This is the module you specifically called out, and it is conceptually the most different from a “drawn” overlay: **it contains no bitmap at all** — only instructions for arithmetic frame combination, used almost exclusively in **X-Ray Angiography (XA)** and **X-Ray Radiofluoroscopy (XRF)** multi-frame acquisitions to implement **Digital Subtraction Angiography (DSA)**.

5.1 The Clinical Problem It Solves

In an angiography run, you inject contrast and acquire a rapid sequence of frames. Bone, soft tissue, and catheter hardware are visible in *every* frame — but you only care about the contrast-filled vessels. The Mask Module tells the viewer: “*take this frame (or an average of several), call it the mask, subtract it from each contrast frame, and display the result.*” What remains, ideally, is (almost) pure vessel contrast — dramatically improving vascular conspicuity without a second radiation exposure or a separate stored image.

5.2 Where the Module Lives

Context	Module	Section
Baked into the multi-frame XA/XRF Image IOD itself	Mask Module	PS3.3 §C.7.6.10
Applied non-destructively via a Presentation State (does not alter the image, fully reversible/adjustable)	XA/XRF Presentation State Mask Module	PS3.3 §C.11.19
Constrained single-item variant used inside a generic Presentation State	Presentation State Mask Module	PS3.3 §C.11.13

The Presentation-State-based variants are the **modern, preferred** route: the source image stays pristine, and the mask recipe is a separate, editable, re-playable object — exactly the Gen-2→Gen-3 shift described in Section 3.

5.3 Mask Module — Attribute Table (PS3.3 §C.7.6.10, Table C.7-16)

Attribute Name	Tag	Type	Description
Mask Subtraction Sequence	(0028,6100)	1	Top-level sequence; one or more Items, each describing one mask operation
Mask Operation	(0028,6101)	1	Defined Term: <code>NONE</code> , <code>AVG_SUB</code> , <code>TID</code> , <code>REV_TID</code> (see 5.4)
Applicable Frame Range	(0028,6102)	1C	Frame number range(s) over which this mask operation applies. Required if Mask Operation is <code>TID</code> / <code>REV_TID</code>
Mask Frame Numbers	(0028,6110)	1C	Frame number(s) used to build the mask (e.g., the pre-contrast frame)
Contrast Frame Averaging	(0028,6112)	3/1C*	Number of contrast frames averaged together before subtraction
Mask Sub-pixel Shift	(0028,6114)	3	Fractional row/column shift applied to the mask before subtraction (motion compensation)
TID Offset	(0028,6120)	2C	Frame offset used to locate the mask frame in <code>TID</code> / <code>REV_TID</code> modes
Mask Operation Explanation	(0028,6190)	3	Free-text explanation of this particular mask operation
Subtraction Item ID	(0028,9416)	1C	Associates this subtraction item with a per-frame Mask Sub-Pixel Shift functional group item
Mask Selection Mode	(0028,9454)	3	<code>SYSTEM</code> or <code>USER</code> — who chose this mask operation
Recommended Viewing Mode	(0028,1090)	2	<code>SUB</code> (view subtracted) or <code>NAT</code> (native, unsubtracted)

*Type 1C in the Presentation State Mask Module variant when Mask Frame Numbers is multi-valued; Type 3 (optional) in the base Mask Module.

5.4 Mask Operation — Defined Terms Explained Simply

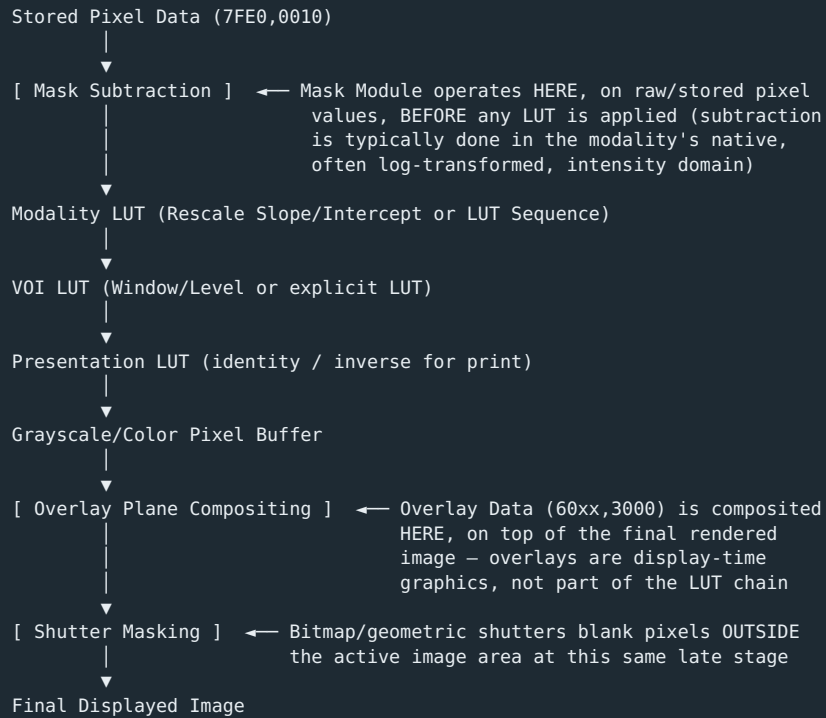
Value	Plain-English Meaning
NONE	No subtraction — display the frame as acquired
AVG_SUB (Average Subtraction)	Average the frame(s) named in Mask Frame Numbers → that average becomes a <i>fixed</i> mask, subtracted from every contrast frame in the applicable range. Classic “one mask for the whole run” DSA.
TID (Time Interval Difference)	Each contrast frame is subtracted from the frame that came <i>N frames earlier</i> (N = TID Offset) — a “sliding” mask that continuously updates, good for reducing motion artifact over a long run
REV_TID (Reverse TID)	Same sliding concept, but computed backward from a starting offset — used when the best mask frame is <i>after</i> the contrast frames in acquisition order

5.5 Practical Walkthrough (AVG_SUB Example)

1. Acquire 30 frames: frames 1–3 are pre-contrast (fills the vessel with nothing but background).
2. Mask Frame Numbers = 1,2,3 , Mask Operation = AVG_SUB → viewer averages frames 1-3 into one synthetic mask image.
3. Applicable Frame Range = 4,30 → subtraction is applied to frames 4 through 30.
4. If the patient moved slightly, Mask Sub-pixel Shift nudges the mask by e.g. (0.4, -0.2) pixels before subtracting, to re-align it (this is the “pixel shift” the user’s notes referenced).
5. Recommended Viewing Mode = SUB tells the viewer to default to showing the subtracted result, not the native frame.

6. Where Overlay/Mask Sit in the Rendering Pipeline

For anyone building or debugging a viewer, the ordering matters enormously. A simplified grayscale rendering pipeline, with overlay/mask insertion points marked:



Key architectural takeaway: Mask (subtraction) is a *pixel-value* operation that happens early, in the acquisition-intensity domain — it changes what number each pixel holds. Overlay is a *compositing* operation that happens last, on the already-windowed, already-color-mapped display buffer — it never changes underlying pixel values, only what's drawn over them. Confusing these two stages is the single most common bug in home-grown DICOM renderers.

7. Burned-In Annotation vs. Stored (Removable) Overlay

This distinction is critical for anonymization, legal/regulatory workflows, and AI training-data curation.

Aspect	Stored Overlay (Gen-2/Gen-3)	Burned-In Annotation
Where it lives	Separate DICOM element (Overlay Data, GSPS, SR)	Inside Pixel Data (7FE0,0010) itself
Reversible?	Yes — delete the element, image is unaffected	No — requires image-processing redaction (blur/black-box), which is destructive to that pixel region
Detected by	(60xx,3000) present / GSPS referencing the image	Burned In Annotation (0028,0301) = YES
Typical source	PACS/viewer-added ROI, angio consoles, AI overlays	Ultrasound consoles, ECG-strip captures, some legacy CR/CT scanner “screen saves”, scanned film digitized with a label baked in
Anonymization approach	Strip/zero the element (cheap, safe, complete)	Requires pixel-level PHI detection (OCR/CV) + inpainting or black-box masking — never 100% guaranteed without manual QA
Regulatory relevance	Lower risk element to audit	High-risk: a mis-flagged or missed burned-in annotation is one of the most common causes of PHI leakage in de-identified research datasets

7.1 Key Attribute for This Distinction

Attribute Name	Tag	Type	Values / Meaning
Burned In Annotation	(0028,0301)	3	YES / NO — declares whether Pixel Data contains burned-in patient-identifying text/graphics
Recognizable Visual Features	(0028,0302)	3	YES / NO — whether the image itself (e.g., a face in a CT surface render) is recognizable, independent of text
Longitudinal Temporal Information Modified	(0028,0303)	3	Declares whether dates/times were shifted during de-identification

7.2 Anonymization / De-identification Overlay Masking in Practice

DICOM PS3.15 defines standard **De-identification Profiles** and **Options**. When applied to overlays specifically, a compliant de-identification pipeline should, at minimum:

- Remove Overlay Data (60xx,3000)** and all sibling overlay attributes in the 60xx repeating group, unless explicitly retained and verified free of PHI (e.g., a pure geometric shutter).
- Check Burned In Annotation**. If YES (or unknown/absent and the modality is known to burn in text — classically ultrasound), route the image through pixel-level redaction (black-box/blur over detected text regions) *before* it leaves a trusted zone — the DICOM header alone cannot fix this.
- Strip or edit GSPS/SR objects** that reference the image and may themselves carry annotation text with patient identifiers (measurement labels, free-text annotations).
- Re-flag** Burned In Annotation = NO and Patient Identity Removed (0012,0062) = YES only after redaction is verified — never as a blind default.

This is precisely the boundary where “DICOM masking overlay” (a standard mechanism) and “anonymization masking” (a security/compliance workflow that *uses* standard mechanisms plus image processing) meet — and where most real-world PACS/VNA anonymization tooling earns its complexity.

8. How Commercial PACS/Diagnostic Viewers Actually Handle This

Generalized, vendor-neutral description of common implementation patterns seen across commercial and open-source viewers (OsiriX/Horos-class, enterprise PACS diagnostic viewers, vendor XA/cath-lab review stations):

- **Overlay Plane Module (60xx) support** is nearly universal for *display* (most viewers will render Overlay Type `G / R` bitmaps in a configurable color, often green or yellow, with an on/off toggle in the UI). Support for *creating/editing* overlays back into the object is rarer — most modern tools prefer writing a GSPS or annotation object instead of round-tripping 60xx groups, because 60xx is display-only and not well indexed by PACS databases.
- **Shutter handling** (both geometric and bitmap/overlay-based shutters) is almost always applied automatically and silently — clinicians typically never see it as a separate toggle; it's baked into “correct” image display.
- **Mask/DSA subtraction** is a first-class, dedicated feature in angiography/cath-lab review workstations — typically exposed as a “Subtracted / Native” toggle plus manual “Pixel Shift” or “re-mask” tools letting the technologist manually correct for patient motion, directly manipulating the equivalent of `Mask Sub-pixel Shift`. General-purpose diagnostic viewers (radiology PACS not focused on XA) often only partially support this or defer to the modality/vendor's own review application.
- **GSPS-based overlays/annotations** are increasingly preferred by modern viewers because they are queryable (C-FIND can locate a Presentation State by Referenced SOP Instance), non-destructive, and support richer graphic primitives (polylines, text, compound graphics) than the binary Overlay Plane Module.
- **Segmentation (SEG) and Parametric Map objects** are the modern replacement for “ROI-type overlays” in AI-driven workflows — an AI contouring tool today virtually always emits a Segmentation IOD (with per-voxel/per-pixel labeled data and semantic coding) rather than a 60xx ROI overlay, because SEG objects carry proper anatomic/finding coding (SNOMED/DICOM coded concepts) and multi-label support that a 1-bit overlay cannot.
- **Burned-in text handling**: most enterprise de-identification/anonymization products (embedded in VNA ingest pipelines or standalone anonymizer appliances) combine (a) DICOM tag-based scrubbing with (b) an OCR/CV “burned-in text detector” specifically because vendors know Overlay/GSPS stripping alone is insufficient — this is usually a distinctly licensed/configured module, reflecting how seriously the industry treats this specific gap.
- **Legacy Gen-1 embedded bit-plane overlays**: virtually no modern viewer actively supports rendering these (they're retired); the practical risk is silent data loss during transcoding of old studies (e.g., recompressing to JPEG or converting Bits Allocated) rather than a rendering bug — worth flagging specifically in any legacy-data migration or VNA ingestion project.

9. Use Cases Summary

Use Case	Mechanism	Why
Cath-lab vessel visualization (DSA)	Mask Module (AVG_SUB/TID/REV_TID)	Removes static anatomy, isolates contrast
Ultrasound sector/cone active-area display	Shutter Overlay	Hides scan-converter padding pixels
Manual ROI drawn by a radiologist and saved with the study	Overlay Plane Module (R type) or GSPS	Legacy simple case vs. modern preferred case
AI-generated tumor segmentation	Segmentation IOD (not classic overlay)	Needs semantic labeling, multi-class, volumetric support
Biopsy needle path graphic burned by console	Overlay Plane Module (G type) or burned-in pixel (console-dependent)	Vendor-dependent; some consoles still burn-in
Research dataset de-identification	PS3.15 profile + overlay/GSPS stripping + burned-in text redaction	Regulatory (HIPAA/GDPR) PHI removal
Print/film composite with permanent annotation	Burned-in annotation	Legal record requiring immutable annotation
Cross-vendor annotation exchange (measurements, findings)	Structured Report (SR) + GSPS	Machine-readable, queryable, standards-based interoperability

10. Code Examples

10.1 Reading a Classic Overlay Plane with `pydicom`

```
import pydicom
import numpy as np

ds = pydicom.dcmread("angio_with_overlay.dcm")

# Overlay group 6000 (the first of up to 16 possible groups: 6000,6002,...601E)
group = 0x6000
rows    = ds[group, 0x0010].value      # Overlay Rows
cols    = ds[group, 0x0011].value      # Overlay Columns
otype   = ds[group, 0x0040].value      # 'G' or 'R'
origin  = ds[group, 0x0050].value      # [row, col]
raw     = ds.overlay_array(group)      # pydicom convenience: unpacks the bitmap

print(f"Overlay type={otype}, size={rows}x{cols}, origin={origin}")
print(f"Overlay pixels on: {np.count_nonzero(raw)}")
```

`pydicom` provides `Dataset.overlay_array(group)` specifically because unpacking the 1-bit-per-pixel Overlay Data (60xx,3000) by hand (bit-order, row-major packing) is error-prone — always prefer the library helper over manual bit-twiddling.

10.2 Interpreting the Mask Module with `pydicom`

```
import pydicom

ds = pydicom.dcmread("dsa_run.dcm")

mask_seq = ds.MaskSubtractionSequence # (0028,6100)
for item in mask_seq:
    op = item.MaskOperation              # (0028,6101)
    frame_range = getattr(item, "ApplicableFrameRange", None)
    mask_frames = getattr(item, "MaskFrameNumbers", None)
    avg = getattr(item, "ContrastFrameAveraging", None)
    shift = getattr(item, "MaskSubPixelShift", None)
    tid_offset = getattr(item, "TIDOffset", None)

    print(f"Operation={op}, range={frame_range}, mask_frames={mask_frames}, "
          f"avg={avg}, shift={shift}, tid_offset={tid_offset}")
```

10.3 Minimal Manual Subtraction (AVG_SUB) in NumPy

```
import numpy as np

def avg_sub_dsa(frames: np.ndarray, mask_frame_idx: list[int],
                contrast_start: int, contrast_end: int) -> np.ndarray:
    """
    frames: array shaped (n_frames, rows, cols), raw/native intensity domain
    mask_frame_idx: 0-based indices of frames to average into the mask
    contrast_start/end: 0-based inclusive frame range to subtract from
    """
    mask = frames[mask_frame_idx].mean(axis=0)
    contrast = frames[contrast_start:contrast_end + 1]
    subtracted = contrast.astype(np.float32) - mask.astype(np.float32)
    return subtracted

# Example: frames 0-2 are pre-contrast mask, subtract across frames 3-29
result = avg_sub_dsa(frames, mask_frame_idx=[0, 1, 2],
                    contrast_start=3, contrast_end=29)
```

Note: Real implementations must respect Pixel Intensity Relationship (0028,1040) — if pixel values are not already logarithmic, a log transform (per Pixel Intensity Relationship LUT Sequence, tag (0028,9422)) is required *before* subtraction for the arithmetic to correspond to true contrast attenuation.

10.4 Reading an Overlay with `fo-dicom` (C#/.NET)

```

using FellowOakDicom;
using FellowOakDicom.Imaging;

var file = DicomFile.Open("angio_with_overlay.dcm");
var dataset = file.Dataset;

// Group 0x6000 = first overlay plane
var overlayTagRows = new DicomTag(0x6000, 0x0010);
var overlayTagCols = new DicomTag(0x6000, 0x0011);
var overlayTagType = new DicomTag(0x6000, 0x0040);
var overlayTagData = new DicomTag(0x6000, 0x3000);

int rows = dataset.GetSingleValue<ushort>(overlayTagRows);
int cols = dataset.GetSingleValue<ushort>(overlayTagCols);
string type = dataset.GetSingleValue<string>(overlayTagType);
byte[] overlayBytes = dataset.GetValues<byte>(overlayTagData);

Console.WriteLine("Overlay {type}: {rows}x{cols}, {overlayBytes.Length} packed bytes");

```

10.5 Checking Burned-In Annotation Before De-identification (`pydicom`)

```

burned_in = getattr(ds, "BurnedInAnnotation", "NO") # default assume NO if absent – but flag for review
if burned_in == "YES" or ds.Modality == "US":
    # Route to pixel-level redaction pipeline (OCR/CV based) BEFORE
    # relying on tag-based de-identification alone.
    flag_for_pixel_redaction(ds)

```

11. Quick-Reference Comparison: Legacy vs. Modern

Dimension	Legacy (Gen-1/Gen-2)	Modern (Gen-3)
Storage location	Pixel Data high bits (retired) / 60xx group	Separate IOD: GSPS, SEG, SR, Mask via Presentation State
Bit depth	1-bit only	Arbitrary (SEG supports labeled multi-class)
Semantics	None (just “on/off” pixels)	Coded concepts (SNOMED CT), measurement units, findings
Queryable via PACS/DB	No (opaque binary blob)	Yes (Presentation State references, SR content)
Editable/reversible	Fragile, viewer-specific	Designed to be non-destructive and editable
Multi-vendor interoperability	Poor-to-moderate (rendering undefined without a PS)	Strong (explicit IODs, conformance-tested)
Still actively emitted today	Yes, especially US/XA/legacy CR	Yes, increasingly preferred for new integrations & AI output
Max count per image	16 overlay planes (60xx)	Effectively unlimited (sequence-based)

12. References

1. DICOM PS3.3 — *Information Object Definitions*, Section C.9.2 “Overlay Plane Module” and Section A.26.4/A.28.4. NEMA.
2. DICOM PS3.3 — *Information Object Definitions*, Section C.7.6.10 “Mask Module” (Table C.7-16).
3. DICOM PS3.3 — *Information Object Definitions*, Section C.11.19 “XA/XRF Presentation State Mask Module.”
4. DICOM PS3.3 — *Information Object Definitions*, Section C.11.13 “Presentation State Mask Module.”
5. DICOM PS3.3 — *Information Object Definitions*, Section C.7.6.16 “Multi-frame Functional Groups Module” (Frame Pixel Shift Macro context).
6. DICOM PS3.5 — *Data Structures and Encoding*, Section 8.1.2 “Overlay Data Encoding of Related Data Elements,” and “Repeating Groups” (60xx numbering rules).
7. DICOM PS3.15 — *Security and System Management Profiles*, Annex E “De-identification of DICOM Data” (Burned-In Annotation, Recognizable Visual Features handling).
8. DICOM PS3.17 — *Explanatory Information*, Annex FFF “XA/XRF Presentation State Storage” (Mask Module usage recommendations).
9. DICOM PS3.3 — Grayscale/Color Softcopy Presentation State IODs (GSPS/CSPS) — modern annotation/overlay alternative.
10. DICOM PS3.3 — Segmentation IOD (SEG) and Parametric Map IOD — modern ROI/mask alternatives for AI output.
11. `pydicom` documentation — `Dataset.overlay_array()` , overlay and pixel data handling. <https://pydicom.github.io>
12. `fo-dicom` documentation — DicomDataset overlay and tag access patterns. <https://fo-dicom.github.io>

This guide reflects the current (non-retired) DICOM Standard as published by NEMA/DICOM Standards Committee. Always verify against the current-year PS3.3/PS3.5/PS3.15 edition (dicomstandard.org) for conformance statements and product development, since Defined Terms and conditional requirements are periodically refined via Change Proposals (CPs).